

Approach

The answer is **always computed by a deterministic symbolic solver**, never by a language model. The LLM (**Qwen2.5-3B-Instruct**, Qwen License, <= 8B, self-hosted via Ollama, OpenAI-compatible /v1/*) does three utility roles: (1) extract quantities from prose when regex misses, (2) translate NL claims to FOL for Z3, (3) phrase the explanation. Every numeric result comes from SymPy/pint; every logical verdict from forward-chaining or Z3. The explanation is generated from the solver’s trace, so it cannot diverge from the answer – directly serving P2 (explanation) and P3 (reasoning depth).

Pipeline

- **Router**: presence of **premises-NL** -> logic, else physics.
- **Physics**: cleaner -> topic classifier (ID-prefix + keyword + symmetric field/force guards + state-change + symbol fallback) -> 6-pass role-aware regex extractor (chain equality, noun-of-value, “X cm away/apart”, “has resistance / is at”, “draws X A”, ratio) with LLM fallback (N=3 self-consistency + TF-IDF RAG; unit pre-convert forbidden; “x 10^N” scientific peel) -> pint normalisation (cm2/mm3 / turns/m auto-fixed) -> **37-formula YAML registry** (capacitor/Coulomb/plate variants, E-field point/midpoint/perp-bisector, isocetes & right-triangle apex, voltage capacitor distance ratio Q-conserved, magnetic flux/solenoid/wire, RLC impedance/resonance/factor/power, Ohm’s law, passthrough state-change) -> SymPy compute -> sanity verifier -> trace explanation.
- **Logic**: TF-IDF premise selector (R@5 = 81.3% vs the dataset’s gold **idx** field) -> rule parser -> forward chainer -> **Z3** entailment (multi-variable universals, comparison literals) with a NL->FOL parser/vocabulary rejection-and-retry loop (Slide-27 endorsed) -> per-option entailment for MC with **Unknown** abstention.
- **Self-correction** ON: solver re-checks the LLM draft, feedback is *solver-grounded* (never invents numbers/premises), <= 2 rounds. Output schema {**answer**, **explanation**, **cot**, **premises**, **fol**, **confidence**}; malformed JSON impossible (Pydantic); unreachable LLM degrades to the deterministic answer (never a 500).

Training (ablation, not deployed)

QLoRA-SFT on Qwen2.5-7B (unsloth/qwen2.5-7b-instruct, rank 16 alpha 32, 2 epochs, Colab T4, 1,765 pairs from 2026-05-09). Measured on a **163-row SFT-unseen** holdout of 2026-05-15 (scripts/build_sft_unseen_holdout.py fixes a 99% leakage in our prior holdout): single SFT-7B = 24.5% (-6.6pp LD regression, 5x latency); hybrid LD->3B + rest->SFT-7B = 30.1%. **Not deployed**: Q3 “one model resident” compliance + vLLM-LoRA stack is 1-2 days of risk that does not justify marginal lift over deterministic 39.3%.

Data & compliance

Only the official EXACT2026 release **2026-05-15**. No external/synthetic/crawled data, no GPT/Claude/Gemini in the pipeline. See docs/data_disclosure.pdf. 268 automated tests; ruff + mypy CI; 32 ADRs documenting every architectural decision and measurement.

Internal results (163-row SFT-unseen clean holdout, frozen unit-/round-aware scorer)

Variant	Physics Full	Logic correct
Day-1 rule-only / Day-21 honest baseline	0.8% / 22.1%	35.9% / 22.2%
+ F1+F2 audit + field-vs-force / F3+E5 selector+FOL loop	27.6%	23.5%
+ Iter-3 architectural fixes (role-aware regex, state-change, boundary)	33.1%	–
+ Iter-4 routing/unit cleanup (cm2, scientific peel, perp-bisector field)	36.8%	–
+ Iter-5 routing precedence + turns/m alias + Q*U/2 formula	39.3%	–

Iter-5 prefix lift (vs 36.8%): CH 31->38%, TD 50->54%; others flat. Cumulative vs 27.6%: +11.7pp deterministic, no SFT. Physics correctness uses strict value-and-unit match (pint dimension + 1% rel tolerance OR round-aware at gold’s stated precision). The deterministic solver is exact where extraction succeeds; residual = LLM extraction recall on hard multi-step. Logic is translator-bound on 3B; symbolic Z3 core sound.

Deployment

docker compose -f docker/docker-compose.yml up -d -> GPU Ollama sidecar (qwen2.5:3b resident, OLLAMA_KEEP_ALIVE=-1) + CPU FastAPI; always-on with /healthz, cron pre-warm. Reproduce: uv run pytest -q; uv run python scripts/run_eval.py --task {physics,logic} --with-llm --split data/official_v20260515/eval_split/<task>_eval.jsonl.